Practical Work No. 9
Pointers

I. Definition

A pointer is a variable or constant whose value is the memory address of an object. The address of an object depends on its type. For primitive types, we can define character pointers, integer pointers, and float pointers.

II. Declaration

A pointer is declared using the type of the object being pointed to, preceded by the indirection operator (symbol `*`).
Syntax:

```
Type_Object_Pointed *Pointer_Name;
```

Exemples :

```
char *pc; /* pc is a pointer pointing to an object of type char */
int *pi;  /* pi is a pointer pointing to an object of type int */
float *pf; /* pf is a pointer pointing to an object of type float */
```

III. Assigning an Object's Address to a Pointer

The address of an object is assigned to a pointer using the address-of operator (`&`), as per the syntax: `Pointer = &Object;`

Examples:

```
int i = a, *pi;
pi = &a;
```

Note: You cannot directly assign a value to a pointer. The following code is not allowed:

```
int *pi;
pi = 0xfffe; /* Error */
```

However, you can define the value of an address using the cast operator.
Example:

```
int *pi;
pi = (int*)0xfffe; /* pi points to the address 0xfffe */
```

IV. Assigning the Value of a Variable Through a Pointer

The value of an object can be assigned through a pointer using the indirection operator (`*`), as per the syntax:

```
*Pointer = Value ; /*or Expression; */
```

Example:

```
int x, *p;
...
p = &x;
*p = 34;
...
```

V. Pointer Arithmetic

Pointers can be moved within the memory using addition, subtraction, increment, and decrement operators. The movement is limited to memory blocks that are multiples of the size of the type to which the pointer refers (`n * sizeof(type)`).

Example:

```
int T[10], *p;
...
p = &T[4];        /* p points to the cell with index 4 */
*p = 44;          /* T[4]=44 ;  */
...
*(p + 3) = 55;    /* T[7]=55  */
p++;              /* p= &T[5]  */
```

VI. Using scanf with Pointers

To input a character or a number, the `scanf` function receives the address of the variable. In the case of pointers, only the name of the pointer is passed to `scanf`.

Example:

```
float *pf;
…
scanf("%f", pf);
```

VII. Application Exercises

Exercise 1
This exercise is somewhat meaningless but is meant to familiarize you with pointers of primitive types.

1. Define a variable var of a primitive type.
2. Declare two pointers, pVar1 and pVar2, of the same primitive type.
3. Set pVar1 to point to var.
4. Add 2 to var using pVar1 (not var).
5. Assign pVar1 to pVar2.
6. Add 5 to the variable pointed to by pVar2.
7. Display the addresses of the three variables, their values, and the values pointed to by pVar1 and pVar2.

Exercise 2

Given the following C program:

```
void main() {
    int A = 1, B = 2, C = 3;
    int *P1, *P2;
    P1 = &A;
    P2 = &C;
    *P1 = (*P2)++;
    P1 = P2;
    P2 = &B;
    *P1 -= *P2;
    ++*P2;
    *P1 *= *P2;
    A = ++*P2 * *P1;
    P1 = &A;
    *P2 = *P1 /= *P2;
}
```

Complete the following table for each instruction in the program:

Instruction	**A**	**B**	**C**	**P1**	**P2**
Initialization	1	2	3	-	-
P1=&A ;	1	2	3	&A	-
P2=&C ;					
*P1= (*P2)++ ;					
P1=P2 ;					
P2=&B ;					
*P1-=*P2;					
++*P2;					
P1=*P2;					
A=++*P2**P1;					
P1=&A;					
*P2=*P1/=*P2;					